

TRAC-Tel: Fleet Telemetry & SIM Balance Management System

System Requirements Specification (SRS) & Technical Architecture

Oulmi Ayyoub

IT Access Control Administrator
UTE TR-SE Hassi Messaoud Project

September 2026

Abstract

This document outlines the System Requirements Specification (SRS) and Architectural Blueprint for **TRAC-Tel**, an edge telemetry and SIM data quota management system designed for Honeywell ScanPal EDA52 access control terminals deployed at the UTE TR-SE Hassi Messaoud refinery project. The system operates as a zero-cost, standalone telemetry platform that provides real-time cellular data tracking, dynamic remote USSD execution, and terminal hardware health monitoring across remote site gates over Ooredoo 4G infrastructure.

Contents

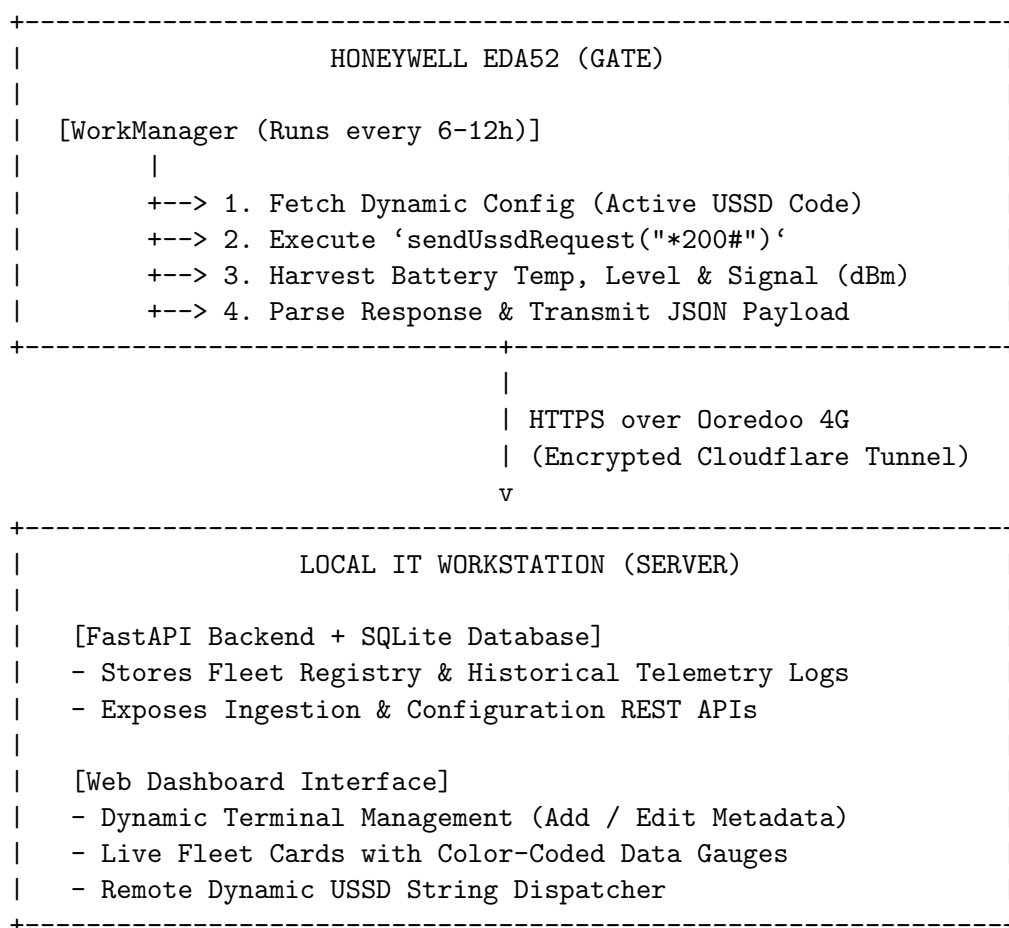
1	System Overview & Operational Context	3
2	Zero-Cost Network Topology (Cloudflare Tunnel)	3
3	Technology Stack	4
4	Functional Requirements	4
4.1	Module A: Web Management Dashboard	4
4.2	Module B: Ingestion & Backend REST API	5
4.3	Module C: Android Edge Telemetry Agent	5
5	Database Schema Specification	5
6	Non-Functional Requirements (NFR)	6
7	Implementation Roadmap	6

1 System Overview & Operational Context

The UTE TR-SE Hassi Messaoud Project operates multiple access control gate points equipped with Honeywell ScanPal EDA52 rugged mobile terminals (running Android 11 in locked kiosk mode). These terminals rely on commercial Ooredoo 4G SIM cards to synchronize personnel and vehicle access transactions with the central ATLAS platform.

Problem Statement: Because terminals operate autonomously at remote perimeter gates, manual verification of SIM data consumption (via interactive USSD codes such as *200#) is inefficient, error-prone, and poses the operational risk of terminals going offline unannounced due to exhausted data quotas.

Solution: **TRAC-Tel** introduces an automated, decoupled edge telemetry client running silently on each EDA52 terminal. The agent executes scheduled USSD balance queries, harvests hardware telemetry (battery level, battery temperature, 4G signal strength), and transmits telemetry payloads over Ooredoo 4G to an on-premise IT server via an encrypted Cloudflare Tunnel.



2 Zero-Cost Network Topology (Cloudflare Tunnel)

To guarantee zero recurring cloud hosting costs while retaining real-time communication across the Ooredoo 4G cellular network, TRAC-Tel utilizes a secure reverse tunnel architecture:

1. **Host-Side Connector (cloudflared):** A lightweight, daemon runs on the local IT workstation. It establishes persistent outbound TLS connections to Cloudflare’s global edge network.

2. **Ingress Routing:** Traffic directed to a provisioned endpoint (e.g., `https://trac-tel-hassi.internal`) is routed directly to the local FastAPI server listening on `localhost:8000`.
3. **Zero Firewall Modification:** No public static IPv4 address, port forwarding, or site router reconfigurations are required.
4. **Edge Client Access:** Honeywell EDA52 terminals submit HTTPS telemetry payloads and fetch configuration updates seamlessly over 4G cellular connections from anywhere on site.

3 Technology Stack

Layer	Technology	Engineering Justification
Edge Client	Android (Kotlin / Java), API 26–30	Native <code>sendUssdRequest()</code> support via <code>TelephonyManager</code> ; low-overhead <code>WorkManager</code> background scheduler.
Backend API	Python 3.10+ (FastAPI)	High-concurrency async ASGI engine; automatic OpenAPI/Swagger documentation generation.
Persistence	SQLite 3 (via <code>SQLAlchemy</code>)	Zero-configuration, zero-maintenance local embedded storage engine; ACID-compliant.
Frontend UI	HTML5 + Tailwind CSS + Vanilla JS	Lightweight, responsive grid architecture; zero complex build steps; served directly by FastAPI.
Secure Tunnel	Cloudflare Tunnel (Free Tier)	Provides secure outbound-only reverse proxying for real-time 4G communication at zero cost.

Table 1: TRAC-Tel Technology Stack Specification

4 Functional Requirements

4.1 Module A: Web Management Dashboard

- **FR-A1 (Fleet Cards Display):** The dashboard landing page shall display a responsive grid of cards representing all registered terminals. Each card shall display:
 - Terminal Identifier (e.g., `AML-001`) and Gate Description.
 - Assigned SIM phone number.
 - Remaining data balance with dynamic color indicators: **Green** ($> 2\text{ GB}$), **Yellow** ($500\text{ MB} - 2\text{ GB}$), **Red** ($< 500\text{ MB}$).
 - Hardware vitals: Battery state, battery temperature ($^{\circ}\text{C}$), and 4G signal strength (dBm).
 - Last synchronization timestamp.
- **FR-A2 (Terminal Registration):** The interface shall provide an intuitive modal dialog enabling administrators to register new terminals by defining: `id`, `gate_name`, `location_desc`, `sim_phone_number`, `total_quota_mb`, and `active_ussd_code`.
- **FR-A3 (Dynamic Terminal & USSD Modification):** Administrators shall have the capability to edit terminal parameters in real-time, including altering the active USSD query string (e.g., transitioning from `*200#` to `*200*1#`).
- **FR-A4 (Historical Audit):** Clicking a card shall reveal historical telemetry trends (data burn rate, temperature logs) spanning the preceding 7 to 30 days.

4.2 Module B: Ingestion & Backend REST API

- **FR-B1** (POST /api/v1/telemetry): Ingests structured JSON telemetry payloads emitted by edge terminals and appends records to the database.
- **FR-B2** (GET /api/v1/terminals/{id}/config): Delivers active operational parameters (current USSD string, polling frequency) to edge agents upon check-in.
- **FR-B3** (GET /api/v1/terminals): Serializes the complete list of registered terminals along with their most recent telemetry snapshot for UI consumption.
- **FR-B4** (POST /api/v1/terminals & PUT /api/v1/terminals/{id}): Exposes administrative CRUD endpoints to create, update, or decommission terminal nodes.

4.3 Module C: Android Edge Telemetry Agent

- **FR-C1 (Background Worker)**: Operates via `Android WorkManager` under strict battery-optimized scheduling, ensuring resilience across device restarts and kiosk session locks.
- **FR-C2 (USSD Query Execution)**: Dynamically retrieves the target USSD string from the server and dispatches queries programmatically using `TelephonyManager.sendUssdRequest()`.
- **FR-C3 (Multi-Lingual Regex Parsing)**: Processes USSD string responses in French, English, or Arabic to reliably extract numerical balance data, normalizing values into integer megabytes (`remaining_mb`).
- **FR-C4 (Hardware Telemetry Collection)**: Interrogates `BatteryManager` to capture battery percentage and core temperature (°C), and queries `SignalStrength` to capture cellular reception (dBm).
- **FR-C5 (Offline Fail-Safe Buffer)**: If network connectivity is lost during an execution cycle, records shall be buffered locally in an embedded SQLite/Room store and flushed upon connection restoration.

5 Database Schema Specification

The underlying data model is implemented in SQLite 3 to maintain a lightweight, zero-administration operational footprint:

Listing 1: TRAC-Tel Relational Database DDL

```
-- Fleet Terminal Registry Table
CREATE TABLE terminals (
    id TEXT PRIMARY KEY,                -- e.g., 'AML-001'
    gate_name TEXT NOT NULL,            -- e.g., 'Gate A - Main Entry'
    location_desc TEXT,                 -- Physical location notes
    sim_phone_number TEXT,              -- Assigned Ooredoo SIM MSISDN
    total_quota_mb INTEGER DEFAULT 10240, -- Baseline data quota (MB)
    active_ussd_code TEXT DEFAULT '*200#', -- Dynamic USSD string
    poll_interval_hours INTEGER DEFAULT 6, -- Background polling period
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- Historical Telemetry Logs Table
CREATE TABLE telemetry_logs (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    terminal_id TEXT NOT NULL,
    raw_ussd_response TEXT,
```

```
parsed_balance_mb REAL,  
battery_level INTEGER,  
battery_temp_c REAL,  
signal_strength_dbm INTEGER,  
recorded_at DATETIME NOT NULL,  
received_at DATETIME DEFAULT CURRENT_TIMESTAMP,  
FOREIGN KEY (terminal_id) REFERENCES terminals(id) ON DELETE CASCADE  
);  
  
CREATE INDEX idx_telemetry_terminal_date ON telemetry_logs (terminal_id,  
recorded_at);
```

6 Non-Functional Requirements (NFR)

- **NFR-1 (Zero Cost Constraint):** The entire server-side architecture shall run strictly on local on-premise hardware utilizing open-source frameworks and free-tier infrastructure.
- **NFR-2 (Minimal Network Overhead):** Telemetry payloads shall remain below 1.5 KB per transmission to minimize cellular bandwidth usage.
- **NFR-3 (Thermal Awareness):** The dashboard shall highlight visual warnings if terminal battery temperatures exceed 45.0 °C due to Hassi Messaoud ambient desert conditions.
- **NFR-4 (High Availability):** The FastAPI application shall maintain automatic database reconnects and graceful error recovery in the event of unexpected local workstation restarts.

7 Implementation Roadmap

1. **Phase 1 (SRS & Architecture):** Requirement specification and architectural definition.
[Status: Approved]
2. **Phase 2 (Backend Engine & DB):** Implementation of FastAPI server, SQLite schema initialization, and REST endpoints.
3. **Phase 3 (Web UI Dashboard):** Construction of responsive terminal cards, Add/Edit modal dialogs, and real-time state visualizers.
4. **Phase 4 (Android Edge Agent):** Development of the Kotlin background service utilizing `WorkManager`, USSD execution, and hardware metric harvesting.
5. **Phase 5 (Tunnel Setup & End-to-End Field Validation):** Cloudflare Tunnel configuration, physical terminal deployment, and live Ooredoo 4G verification.